in more detail soon. Still, you can consider a policy as representing the requirements that need to be satisfied to determine whether the current user is authorized to access the requested endpoint.

For relatively simple policies, you can use `AuthorizeOptions` within the `AddAuthorization` method to configure a role-based policy. The `AddPolicy` method takes the name of the policy and an `AuthorizationPolicyBuilder`, which has a `RequireRole` method, enabling you to state which roles are required.

Listing 10.14 Configuring a role-based policy in Program.cs

```
builder.Services.AddAuthorization(options =>
{
    options.AddPolicy("AdminPolicy",
    ➥ policyBuilder => policyBuilder.RequireRole("Admin"));
});
```

Having configured the policy named `AdminPolicy`, you can apply it to the `AuthorizeFolder` method to ensure only members of the `Admin` role can access the content:

```
builder.Services.AddRazorPages(options => {
    options.Conventions.AuthorizeFolder("/RolesManager", "AdminPolicy");
});
```

As I noted at the beginning of this section, roles are useful for relatively simple authorization requirements, where access policies can be applied to groups of users. Users either are members of a role or they aren’t. Roles have a benefit in that they are easy to configure and manage.

If your application’s authorization requirements increase in complexity, you will begin to find an increasingly complex collection of roles difficult to manage. At that point, rather than managing authorization based on